



13. Patrón FSM en Unity

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

¿Qué es una Máquina de Estados Finita?

Una **Finite State Machine (FSM)** es un modelo de comportamiento donde un agente solo puede estar en **un estado activo a la vez** y puede transitar entre estados de forma controlada.

Estado actual

Solo un estado activo en cada momento. Determina el comportamiento del agente en cada frame.

Transición

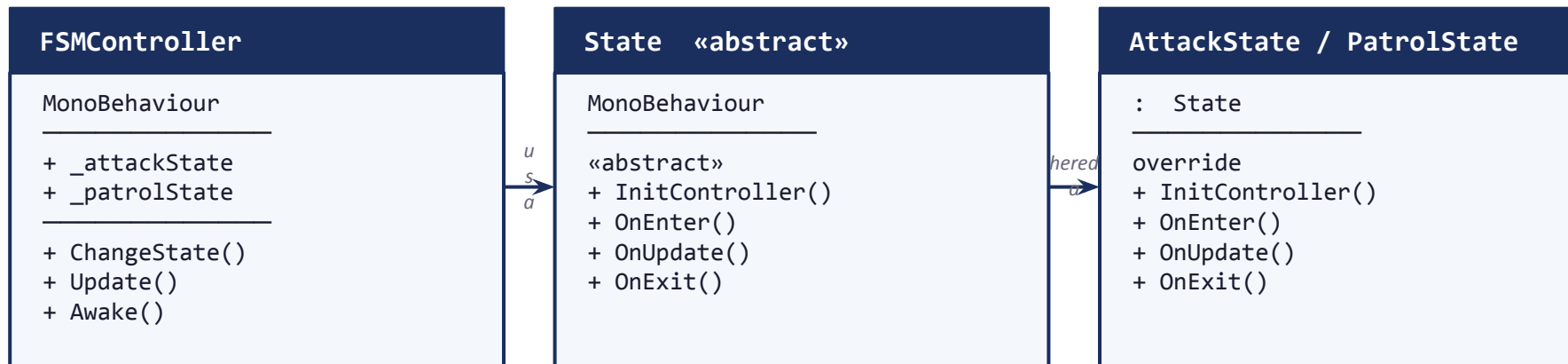
Cambio de un estado a otro.
Puede ocurrir por condiciones de juego (distancia, daño, etc.).

Ciclo de vida

Cada estado tiene 3 fases:
OnEnter (al activarse), OnUpdate (cada frame) y OnExit (al desactivarse).

Las Tres Clases del Patrón

El patrón se compone de **tres tipos de clase**: un controlador que orquesta los estados, una clase base abstracta que define el contrato de cada estado, y los estados concretos que implementan ese contrato.



- FSMController gestiona el estado activo y llama a sus métodos desde Update().
- State define el contrato abstracto: los 4 métodos que todo estado debe implementar.
- Cada estado concreto (AttackState, PatrolState...) hereda de State e implementa su lógica.

La Clase Abstracta State

State es la **clase base abstracta** de todos los estados. Hereda de **MonoBehaviour** para poder adjuntarse como componente al GameObject del enemigo. Define los 4 métodos que todo estado debe implementar.

```
public abstract class State : MonoBehaviour
{
    // Recibe referencia al controlador
    public abstract void InitController(FSMController fsmController);

    // Se ejecuta al entrar en el estado
    public abstract void OnEnter();

    // Se ejecuta cada frame
    public abstract void OnUpdate();

    // Se ejecuta al salir del estado
    public abstract void OnExit();
}
```

InitController()

Se llama una vez en Awake(). Le pasa al estado la referencia al FSMController para que pueda solicitar cambios de estado.

OnEnter()

Se ejecuta una sola vez al activar el estado. Ideal para inicializar variables o reproducir animaciones de entrada.

OnUpdate()

Se llama cada frame desde el Update() del FSMController. Aquí vive la lógica principal del estado.

OnExit()

Se ejecuta una sola vez al desactivar el estado. Ideal para limpiar efectos o parar animaciones.

El FSMController

FSMController es el componente que **orquesta la máquina de estados**: mantiene el estado activo, lo actualiza cada frame e implementa el mecanismo de cambio de estado.

```
public class FSMController : MonoBehaviour
{
    public AttackState _attackState;
    public PatrolState _patrolState;
    private State currentState;

    void Awake()
    {
        _patrolState = GetComponent<PatrolState>();
        _attackState = GetComponent<AttackState>();
        _patrolState.InitController(this);
        _attackState.InitController(this);
        ChangeState(_patrolState);
    }

    void Update()
    {
        currentState?.OnUpdate();
    }
}
```

Referencias públicas a cada estado. Unity las obtiene con `GetComponent<>()` en `Awake()`.

En `Awake()`: obtiene componentes, los inicializa con `InitController(this)` y arranca el estado inicial.

`Update()` delega al estado activo. El operador `?.` evita `NullReferenceException` si no hay estado.

`ChangeState()`: llama `OnExit()` al estado saliente, `OnEnter()` al entrante y actualiza `currentState`.

ChangeState()

ChangeState() garantiza que la transición entre estados siempre ejecute el ciclo de vida completo: primero cierra el estado actual (OnExit) y luego abre el nuevo (OnEnter).

```
public void ChangeState(State newState)
{
    // 1. Cerrar el estado actual
    currentState?.OnExit();

    // 2. Abrir el nuevo estado
    newState?.OnEnter();

    // 3. Actualizar la referencia activa
    currentState = newState;
}
```

Secuencia de transición:

1

PatrolState.OnExit()

Limpia el estado saliente

2

AttackState.OnEnter()

Prepara el estado entrante

3

currentState = attack

Actualiza la referencia

4

AttackState.OnUpdate()

Próximo frame: lógica nueva

Estado Concreto: PatrolState

PatrolState hereda de **State** e implementa los 4 métodos abstractos. Guarda una referencia al controlador para poder solicitar cambios de estado desde **OnUpdate()**.

```
public class PatrolState : State
{
    FSMController myController;

    public override void InitController(
        FSMController fsmController)
    {
        myController = fsmController;
    }

    public override void OnEnter() { }
    public override void OnUpdate()
    {
        // Cuando se cumpla condición:
        myController.ChangeState(
            myController._attackState);
    }
    public override void OnExit() { }
}
```

myController es la referencia al FSMController. Se asigna en InitController() y permite pedir cambios de estado.

InitController() almacena la referencia. Es el único punto de contacto con el controlador al inicializar.

OnUpdate() es donde se comprueban las condiciones de transición. Cuando se cumplan, se llama a ChangeState().

OnEnter() y OnExit() se dejan vacíos en este esqueleto. Se rellenarán con lógica de animación, audio, etc.

Estado Concreto: AttackState

AttackState sigue exactamente la misma estructura que PatrolState. La diferencia será el contenido de sus métodos, no la estructura. El esqueleto es siempre el mismo.

```
public class AttackState : State
{
    FSMController myController;

    public override void InitController(
        FSMController fsmController)
    {
        myController = fsmController;
    }

    // TODO: lógica de ataque
    public override void OnEnter() { }

    // TODO: condición vuelta a Patrol
    public override void OnUpdate() { }

    // TODO: limpieza al salir
    public override void OnExit() { }
}
```

Estructura idéntica a PatrolState:



myController ← referencia al FSM



InitController() ← guardar ref



OnEnter() ← al activarse



OnUpdate() ← cada frame



OnExit() ← al desactivarse

Configuración en el Inspector de Unity

Las tres clases son **MonoBehaviour**, por lo que deben adjuntarse como componentes al mismo GameObject del enemigo. No hace falta arrastrar referencias: **GetComponent<>()** las obtiene automáticamente en **Awake()**.

Enemy (GameObject)
FSMController
PatrolState
AttackState
(otros scripts...)

Los tres componentes deben estar en el mismo GameObject para que **GetComponent<>()** los encuentre.

Pasos para configurarlo:

1

Añadir los tres scripts al mismo GameObject del enemigo.

2

FSMController.Awake() obtiene las referencias con **GetComponent<>()**.

3

Llama a **InitController(this)** en cada estado para pasarles la referencia al controlador.

4

Llama a **ChangeState(_patrolState)** para arrancar la FSM en el estado inicial.